

Pseudo | 

(Task 1 of 6) In this tutorial, we will learn about **mutable values**, illustrated with **vectors**.

The following program illustrates how to create vectors

Pseudo [Run ▶]

```
print(vec[vec[1, 2], vec[3], vec[]]) (mvec (mvec 1 2) (mvec 3) (mvec
```

This program produces `vec[vec[1, 2], vec[3], vec[]]`. It creates four vectors:

- a two-**element** vector that refers `1` and `2`
- a one-element vector that refers `3`
- an empty vector
- a three-element vector that refers all three aforementioned vectors

Pseudo | 

(Task 2 of 6) What is the result of running this program?

Pseudo [Run ▶]

Lispy

```
let n = 44          (defvar n 44)
fun f(x):          (deffun (f x)
  return x + 1      (+ x 1))
end                (mvec n (f n))
print(vec[n, f(n)])
```

`vec[44, 45]`

2

You predicted the output correctly 🎉🎉🎉

3

This program binds `n` to `44` and `f` to a function that adds 1 to its parameter. After that, the program creates a vector that refers the value of `n`, which is `44`, and the value of `f(n)`, which is `45`. The vector is printed as `vec[44, 45]`.

Click [here](#) to run this program in the Stacker.

Pseudo | 

(Task 3 of 6) The following program illustrates how to refer to vector elements.

Pseudo [Run ▶]

JavaScript

```
let v = vec[84, 75]  let v = [ 84, 75 ];
print(v[0])          console.log(v[0]);
print(v[1])          console.log(v[1]);
print(v[2])          console.log(v[2]);
```

This program produces `84 75 error`. It refers to the `0`-th (i.e., first) element, the `1`-th element, and then tries to refer to the `2`-th element.

Pseudo | 

(Task 4 of 6) What is the result of running this program?

Pseudo [Run ▶]

```
let v = vec[vec[58, 43], vec[43, 66]]  (defvar v (mvec (mvec 58 43) (
let vr = v[1]                          (defvar vr (vec-ref v 1))
print(vr[0])                           (vec-ref vr 0))
```

43

6

You predicted the output correctly 🎉🎉🎉

7

Recall that the left-most element is the 0-th (rather than 1-th!) element. `v[1]` produces the value of `vec[43, 66]`. So, `vr[0]` produces 43.

Click [here](#) to run this program in the Stacker.

(Task 5 of 6) The following program illustrates how to mutate vectors.

Pseudo | 🗑️

Pseudo [Run ▶]

Python

```
let m = vec[62, 77]  m = [ 62, 77 ]
m[0] = 83            m[0] = 83
print(m[0])          print(m[0])
```

This program produces 83. It **mutates** the vector by **replacing** the 0-th element with 83 and then refers to the 0-th element.

(Task 6 of 6) What is the result of running this program?

Pseudo | 🗑️

Pseudo [Run ▶]

JavaScript

```
let x = vec[92, 73]  let x = [ 92, 73 ];
x[0] = 67            x[0] = 67;
print(x)             console.log(x);
```

67 73

10

The answer is `vec[67, 73]`.

11

▼ Textual explanation

`x` is bound to a vector. A vector referring 67 and 73 is printed as `vec[67, 73]`.

Click [here](#) to run this program in the Stacker.

What is the result of running this program?

Pseudo | 🗑️

Pseudo [Run ▶]

JavaScript

```
let v = vec[83, 64]  let v = [ 83, 64 ];
```

```
v[0] = 66  
print(v)
```

```
v[0] = 66;  
console.log(v);
```

```
vec[66,64]
```

13

You predicted the output correctly 🎉🎉🎉

14

You have finished this tutorial 🎉🎉🎉

Please the finished tutorial to a PDF file so you can review the content in the future. **Your instructor (if any) might require you to submit the PDF.**

Start time: 1790870357832